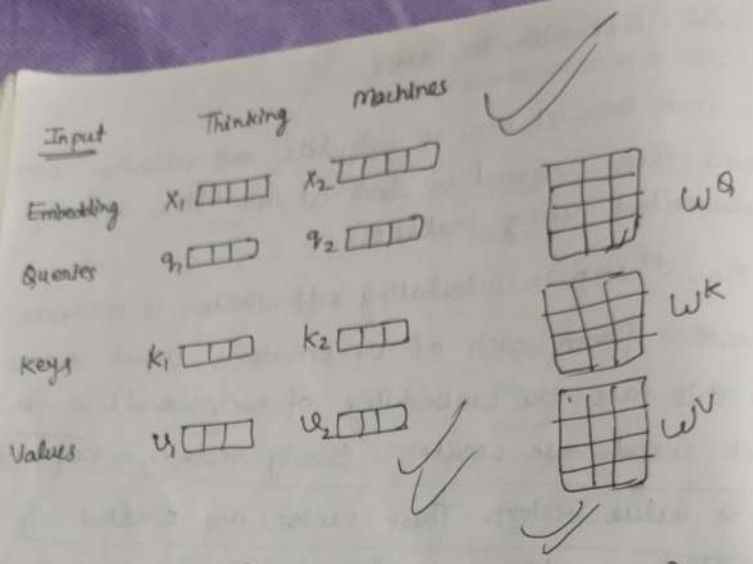
⇒ Self - Attention in Detail

Let's first look at how to calculate self-attention using vectors then proceed to look at how it's actually implemented using matrices.

↳ The 1st step in calculating self-attention is to create 3 vectors from each of the encoder's input vectors (in this case, the embedding of each word). So for each word, we create a Query vector, a Key vector & a Value vector. These vectors are created by multiplying the embedding by 3 matrices that we trained during the training process.

Notice that these new vectors are smaller in dimension than the embedding vector. Their dimensionality is 64, while the embedding & encoder input/output vectors have dimensionality of 512. They don't HAVE to be smaller, this is an architecture choice to make the computation of multiheaded attention (mostly) constant.

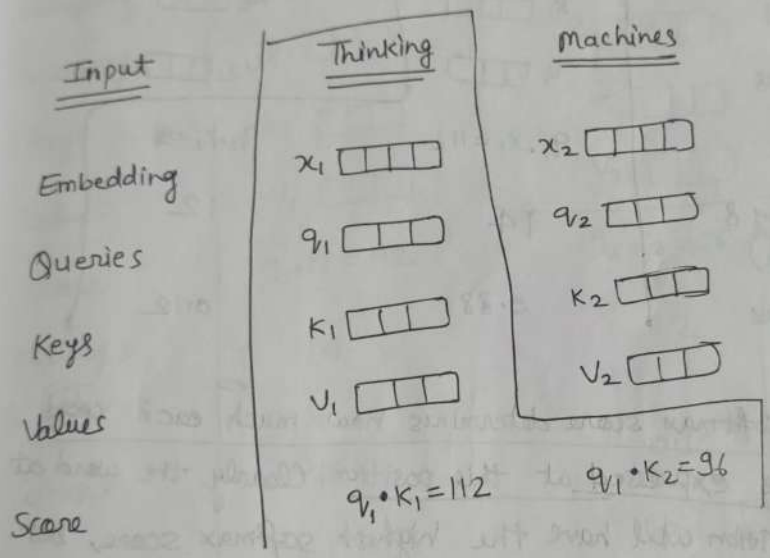


What are the "query", "key" & "value" vectors?

They are abstractions that are useful for calculating and thinking about attention. Once you proceed with reading how attention is calculated below, you'll know pretty much all you need to know about the role each of these vectors play.

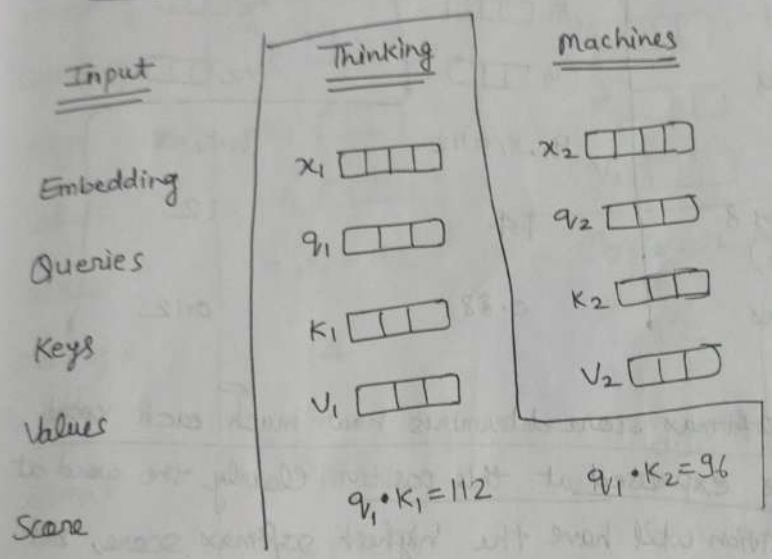
The 2nd step in calculating self-attention is to calculate a score. Say we are calculating the self-attention for the 1st word in this example, "Thinking". We need to score each word of the input sentence against this word. The score determines how much focus to place on other parts of the input sentences as we encode a word at a certain position.

The score is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring. So, if we're processing the self-attention for the word in position #1, the first score would be the dot product of q1 & k1. The second score would be the dot product of q1 & k2.



↳ The 3rd & 4th steps are to divide the score by 8 (The square root of the dimension of the key vectors used in the paper – 64. This leads to having more stable gradients. There could be other possible values here, but this is the default), then pass the result through a softmax operation. Softmax normalizes the scores

The score is calculated by taking the dot product of the query vector with the key vector of the respective word we're scoring. So, if we're processing the self-attention for the word in position #1, the first score would be the dot product of q1 & k1. The second score would be the dot product of q1 & k2.



↳ The 3rd & 4th steps are to divide the score by 8 (The square root of the dimension of the key vectors used in the paper – 64. This leads to having more stable gradients. There could be other possible values here, but this is the default), then pass the result through a softmax operation. Softmax normalizes the scores

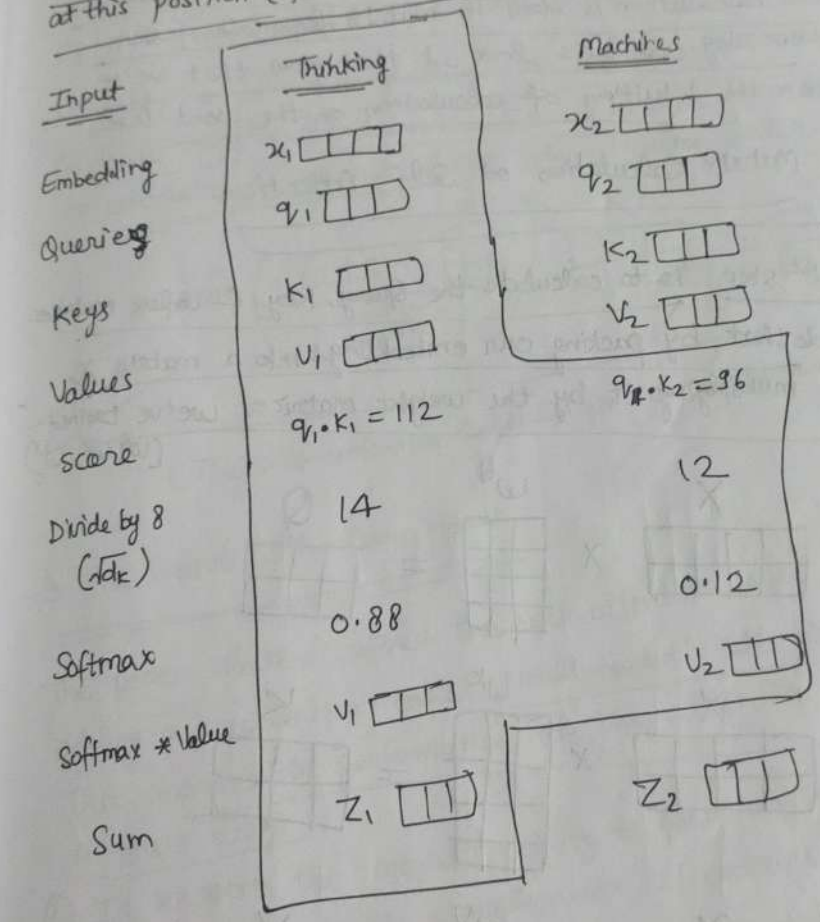
So they are all positive and add up to 1.

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

↳ This softmax score determines how much each word will be expressed at this position. Clearly the word at this position will have the highest softmax score, but sometimes it's useful to attend to another word that is relevant to the current word.

↳ The 5th step is to multiply each value vector by the softmax score (in preparation to sum them up). The intuition here is to keep intact the values of the word(s) we want to focus on, & drown-out irrelevant words (by multiplying them by tiny numbers like 0.001, for example

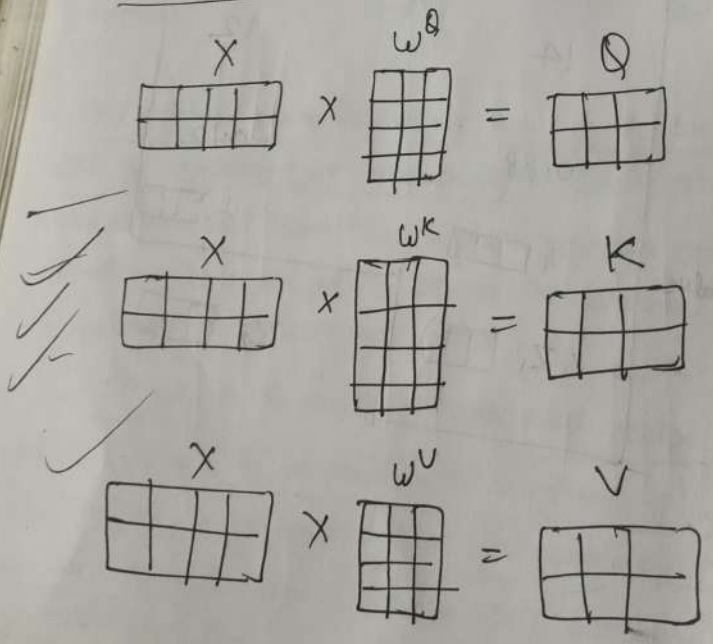
↳ The 6th step is to sum up the weighted value vectors. This produces the output of the ~~sef~~ self-attention layer at this position (for the first word).



That concludes the self-attention calculation. The resulting vector is one we can send along to the feed-forward neural n/w. In the actual implementation, however, this calculation is done in matrix form for faster processing. So let's look at that now that we've seen the intuition of calculation on the word level.

⇒ Matrix Calculation of Self-Attention

The 1st step is to calculate the Query, Key & Value matrices. We do that by packing our embeddings into a matrix X, and multiplying it by the weight matrices we've trained (W^Q, W^K, W^V)



↳ Every row
the input senter
the embeddi
q/K/V vec

→ Finally, si
condense step
to calculate

softma

(Th

⇒ The Beas

The paper
adding a n
This impro
in two
① It expo
position
bit of
by the
sentenc
it was

↳ Every row in the X matrix corresponds to a word in the input sentence. We again see the difference in size of the embedding vector (512, or 4 boxes in figure) and the q/k/v vectors (64, or 3 boxes in the figure).

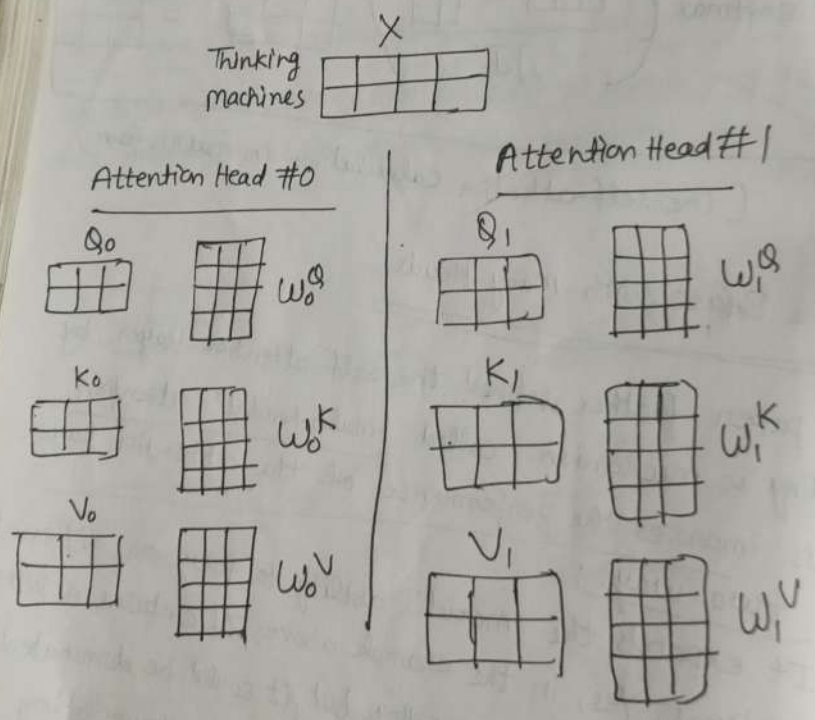
→ Finally, since we're dealing with matrices, we can condense steps two through ~~6~~ in six in one formula to calculate the outputs of the self-attention layer.

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V = Z$$

(The self-attention calculation in matrix form).

⇒ The Beast with Many Heads

The paper further refined the self-attention layer by adding a mechanism called "multi-headed" attention. This improves the performance of the attention layer in two ways:

① It expands the model's ability to focus on different positions. Yes, in the example above, z_1 contains a little bit of every other encoding, but it could be dominated by the actual word itself. If we're translating a sentence like "The animal did not cross the street because it was too tired", it would be useful to know which word "it" refers to.

② It gives the attention layer multiple "representation subspaces". As we'll see next, with multi-headed attention we have not only one, but multiple sets of Query/Key/Value weight matrices (The Transformer uses 8 attention heads, so we end up with 8 sets ~~of~~ for each encoder/decoder). Each of these sets is randomly initialized. Then, after training, each set is used to project the input embeddings (or vectors from lower encoders/decoders) into a different representation subspace.